Algoritmalar ve Veri Yapıları Bootcamp Ödev 3.
Çözümler

1.

Algoritma *Recursive_Binary_Search (A[0..n-1], I[0..n-1], key)*

*//*Algorithm that searches a given key in a given sorted array of n elements.
*//***Input:** An array of n ordered values, two pointers one indicating the left end position, other indicating the right end position of the given array, relevant for the search. A key to be searched in the array.
*//***Output:** The index of the array which contains the key, -1 if the key is not present in the array.

```
if n=1 then
        if A[0] = key then
                return I[0]
        else
                return -1
mid ← ⌊(n-1) / 2⌋
if A[mid] = key then
        return I[mid]
else if key < A[mid] then
        copy B[0..mid-1] ← A[0..mid-1]
        copy II[0..mid-1] ← I[0..mid-1]
        return Recursive_Binary_Search (B[0..⌊n/2⌋-1], II[0..⌊n/2⌋-1], key)
else    // key > A[mid]
        copy B[0..⌈(n-1)/2⌉] ← A[mid+1..n-1]
        copy II[0..⌈(n-1)/2⌉] ←I[mid+1..n-1]
        return Recursive_Binary_Search (B[0..⌊n/2⌋-1], II[0..⌊n/2⌋-1], key)
```

İşleme başlamak için yukarıdaki algoritma şu şekilde çağrılmalıdır:

I[0..n-1] ←[0, 1, 2, 3, 4, 5, 6, ……, n-1] // Örtülü döngü var.
Recursive_Binary_Search (A[0..n-1], I[0..n-1], key)

I dizisi (yardımcı dizi), karşılık gelen ana dizi (A) hücreleri için indeks değerlerini tutar; yani dizinin ilk hücresinin indeks değeri 0, ikinci hücresinin indeks değeri 1, son hücresinin indeks değeri n-1 vb.'dir. Algoritma başladığında her I[i] i'ye eşittir, ancak daha sonra ardışık özyinelemeli çağrılar sonrasında bu ilişki ortadan kalkacaktır. Algoritma dizide aranan anahtarı bulduğunda (eğer varsa tabi), elemanın bulunduğu asıl dizi hücresinin orjinal indeks numarası olarak I dizisinin karşılık gelen hücresindeki değer döndürülecektir.

Zaman Karmaşıklığı Analizi: Algoritmanın zaman karmaşıklığını bulabilmek için "recurrence" denklemini yazabilmeliyiz. Problem boyutumuz n, temel operasyonumuz da karşılaştırma (**if** deyimi) olsun. Algoritmanın her bir çağrımında iki adet karşılaştırma görünüyor. Ancak görünmeyen bir kaç adet daha üstü örtülü döngü (ve her bir döngü için karşılaştırma) satırları mevcut. Her bir copy işlemi aslında bir döngü dahilinde, kopyalanan değer adedi kadar karşılaştırma işlemi içerecektir. Dahası, algoritmanın özyinelemeli çağrımı esnasında da, "call-by-value" esasına dayandığımız için, belli sayıda değerin çağıran noktadan çağrılan noktaya kopyalanması gerekecek, bu da benzer şekilde örtülü bir şekilde kopyalanan değer adedi kadar karşılaştırma işlemi içerecektir. Bu örtülü döngüleri şu şekilde hayal edebilirsiniz:

for i←<*alt_sınır*> **to** <*üst_sınır*> **do**
    B[i] ←A[i]

Yukarıdaki basit döngü <*üst_sınır*> - <*alt_sınır*> değerine eşit sayıda kadar dönerken, döngünün sona erip ermediğine karar verebilmek için toplam aynı sayıda

if i > <üst_sınır> **then**

karşılaştırması yapmaktadır. Dolayısıyla algoritmamız aslında her bir özyinelemeli çevrimde 2 adet görünen karşılaştırma (if), 4xn/2 adet de görünmeyen örtülü karşılaştırma (for dolayısıyla if) işlemi koşturmaktadır. Mevcut else bloklarından sadece bir tanesinin ilerleyeceğini, yani algoritmanın kodunda görünen iki özyinelemeli çağrımdan koşu zamanında sadece bir tanesinin çalışacağını da düşünürsek, algoritmanın zaman karmaşıklığını hesaplayacağımız "recurrence" denklemi şu şekilde ortaya çıkacaktır.

$T(n) = 1*T(n/2) + 2 + 4*n/2$

Master Teoremini baz alarak:

a = 1, b = 2, d = 1 olduğundan hareketle:

$a < b^d$ olduğundan ilk kural geçerli olacaktır ve;

T(n) is O(n) yani lineer çıkacaktır. Yani, algoritma çağrım esasımız "call-by-value" esasına dayanıyorsa, algoritmanın zaman karmaşıklığı bir üst sınıfa çıkmaktadır. Bir başka deyişle, orjinal dizinin tamamı yerine yarısını bir sonraki safhaya geçirmeye çalışmak bize daha pahalıya patlamış görünüyor!

Uzay Karmaşıklığı Analizi: Girdi parametrelerinin uzay karmaşıklığına artırıcı yönde katkı yapmadığını varsayıyoruz. Uzay karmaşıklığı için algoritmanın içinde ilave olarak tanımlanan değişkenleri hesaba katacağız.

Stack segment için hafıza gereksinimini hesaplamamız lazım. Algoritmanın mid ve key isimli (skalar tipteki) değişkenleri için sabit bir hafıza gereksinimi (O(1)) var. Bunun yanı sıra algoritmamız n/2 boyutlu iki adet de extra dizi için (B ve I) hafızada yer ayırmak zorunda.

Özyineleme ağacının yüksekliğinin logn olduğunu düşünürsek, her bir çağrımın ihtiyacı olan 2*O(1) değişkenler için hafıza ihtiyacı, algoritma sonlanıncaya kadar, 2logn kadar bir hafızaya tekabül edecektir. B ve I dizileri için gerekecek olan hafıza miktarı da, algoritma sonlanıncaya kadar şöyle bir seriye karşılık gelecektir.

n + n/2 + n/4 + n/8 + …. + 2 + 1

Bu da eşittir:

n(1 + ½ + ¼ + ⅛ + …… + 1/(n/2) + 1)

Yukarıdaki parantez içerisindeki seri n sonsuza doğru gittiğinde 2 değerine yakınsayacaktır. Dolayısıyla, yerel değişkenler için stack segment'de ayrılması gereken hafızayı düşündüğümüzde, algoritmanın uzay karmaşıklığı:

S(n) = 2logn + 2n

Yukarıdaki terimde, düşük dereceli logn ifadesini düşürebilir ve n'nin başındaki 2 katsayısını göz ardı edebiliriz. Dolayısıyla, algoritmamızın uzay karmaşıklığı (girdilerin uzay karmaşıklığına katkı yapmadığı varsayımı altında)

S(n) is O(n) yani lineer olacaktır.

Özyinelemeli almayan, yani iteratif ikili arama algoritmasının uzay karmaşıklığı, aynı varsayım altında (Girdi parametrelerinin uzay karmaşıklığına artırıcı yönde katkı yapmadığı varsayımı), O(1) yani sabittir. Bir başka deyişle, algoritmayı özyinelemeli bir hale çevirip, her bir özyineleme esnasında dizinin tamamı yerine bir parçasını bir sonraki safhaya geçirmek (bu alt diziyi oluşturmak ek hafıza gereksinimi yarattığından) ve bir de algoritma çağrımlarında "call-by-value" esasına dayanır olmak, zaman karmaşıklığında olduğu gibi, uzay karmaşıklığında da, iki algoritmanın hafıza gereksinimini sabitten lineer "order-of-growth" seviyesine taşımış oldu.

Bu sonuçları biraz daha iyileştirmek adına, “call-by-value” esasından “call-by-reference” esasına geçmeliyiz. Bu öncelikle, alt algoritma çağrımı esnasında gerekli olan kopyalama gereksinimini ortadan kaldıracaktır. Dolaysıyla çağrım esnasındaki örtülü for döngüleri ve gizli karşılaştırmalardan kurtulacağız, zira artık bir algoritmadan bir başkasına, ya da özyinelemeli bir algoritmanın bir koşusundan bir sonraki koşusuna dizi tipindeki değişkenler bütün elemanları kopyalanarak değil, sadece başlangıç adresleri ve eleman adetleri geçirilerek iletileceklerdir. Bu, algoritmanın uzay karmaşıklığına olumlu katkı yaparak S(n)’i logn sınıfına çekecektir.

Ancak, bu iyileştirme sonucunda dahi, algoritmadaki if ve else if bloklarındaki kopyalamalar (özyinelemeli çağrımdan hemen önceki kopyalamalar) halen baki olacağından, zaman karmaşıklığı yine de lineer (***O(n)***) seviyesinde kalacaktır.

2. Merge-sort için en iyi durum, birleştirme (merge) aşamasındaki iki diziden birinin tüm elemanlarının diğer dizinin en küçük elemanından daha küçük (ya da en büyük elemanında daha büyük) olması durumudur. Bu durumda, merge esnasındaki karşılaştırma işlemleri bir dizinin tüm elemanları yarı yolda tükeneceğinden dolayı n/2 karşılaştırmadan sonra bitecek, kalan diğer dizinin elemanları doğrudan karşılaştırma yapmadan hedef diziye kopyalanacak ve birleştirme (merge) işlemi sonlanacaktır (En kötü durumda hiç bir alt dizinin elemanları diğerinden önce tükenmiyor ve son elemanlarına kadar birlikte ilerliyorlar, dolayısıyla merge için yapılması gerek karşılaştırma adedi de n-1 gibi bir değer kadar ulaşıyordu).

Bu da, Merge-sort algoritmasının “recurence” denklemini şu şekle sokacaktır:

$$T(n) = 2*T(n/2) + \mathbf{n/2}, \qquad T(1) = 0$$

Her ne kadar yukarıdaki denklemde + işaretinden sonraki terim biraz azalmış olsa da (en kötü durumda n-1 iken n/2 ye düştü), Master Teoremini uygulama açısından hiç bir şey değişmedi zira hala a=2, b=2 ve d=1. Sonuç olarak algoritmanın zaman karmaşıklığı da değişmemiş oldu:

T(n) is ***O(nlogn)***.

3. Dizi kopyalama işleminin üstü örtülü döngü ve döngünün ne zaman sona ereceğini anlayabilmek için de yine örtülü karşılaştırma içerdiğini yukarıda ilk sorunun cevabında

görmüştük. Şimdi doğrudan temel operasyonumuzu kopyalama olarak kabul ediyoruz. Her bir dizi elemanını hafızada bir yerden diğer bir yere kopyalamak O(1) zaman maliyetine sebep oluyorsa, n adet elemanı birden kopyalamanın maliyeti ya da zaman karmaşıklığı O(n) olacaktır. Merge sort algoritmasını hatırlayacak olursak:

ALGORITHM *Mergesort*$(A[0..n-1])$
//Sorts array $A[0..n-1]$ by recursive mergesort
//Input: An array $A[0..n-1]$ of orderable elements
//Output: Array $A[0..n-1]$ sorted in nondecreasing order
if $n > 1$
copy $A[0..\lfloor n/2 \rfloor - 1]$ to $B[0..\lfloor n/2 \rfloor - 1]$
copy $A[\lfloor n/2 \rfloor..n-1]$ to $C[0..\lceil n/2 \rceil - 1]$
Mergesort$(B[0..\lfloor n/2 \rfloor - 1])$
Mergesort$(C[0..\lceil n/2 \rceil - 1])$
Merge(B, C, A)

Ana algoritmada toplam n adet dizi elemanının (2*n/2) kopyalanma işlemi var. Merge algoritmasını hatırlayacak olursak:

ALGORITHM *Merge*$(B[0..p-1], C[0..q-1], A[0..p+q-1])$
//Merges two sorted arrays into one sorted array
//Input: Arrays $B[0..p-1]$ and $C[0..q-1]$ both sorted
//Output: Sorted array $A[0..p+q-1]$ of the elements of B and C
$i \leftarrow 0;\ j \leftarrow 0;\ k \leftarrow 0$
while $i < p$ **and** $j < q$ **do**
if $B[i] \leq C[j]$
$A[k] \leftarrow B[i];\ i \leftarrow i + 1$
else $A[k] \leftarrow C[j];\ j \leftarrow j + 1$
$k \leftarrow k + 1$
if $i = p$
copy $C[j..q-1]$ to $A[k..p+q-1]$
else copy $B[i..p-1]$ to $A[k..p+q-1]$

Buradaki kopyalama işlemi ya if ya da else bloğunda yapılıyor ve toplamda, en kötü durumda n/2 adet değer kopyalanıyor (Bunun en kötü durumunun bir önceki soruda, karşılaştırma işlemini temel operasyon kabul ettiğimizdeki en iyi duruma denk geldiğine dikkat!). Sonuç olarak merge-sort algoritmamızın “recurrence” denkleminin aldığı hal:

T(n) = 2*T(n/2) + n/2 + n/2 + n/2

Yukarıdaki denklemde ilk iki n/2 terimi Merge alt algoritması çağrılmadan önceki iki toplu kopyalama adımını, son n/2 terimi de Merge'ün son aşamasındaki tek toplu kopyalama aşamasını temsil ediyor. Hatırlayalım, artık karşılaştırm (**if**) satırlarını değil, kopyalama işlemlerini temel operasyon olarak kabul ediyoruz.

$$T(n) = 2*T(n/2) + 3n/2$$

Yukarıdaki denklemde, Master Teoremini uygulamak için bize lazım olanlar:

$$a = 2, b= 2, d = 1$$

Gördüğümüz gibi, a, b, ve d değerleri, algoritmanın temel operasyonu karşılaştırma olarak kabul ettiğimiz durumdan farklı çıkmadı ve yine Master Teoreminin ikinci kuralı uygulanmalı. Yani;

$T(n)$ is $O(n\log n)$ yani lineeritmik, değişen bir şey olmadı.

Esasen temek operasyonlarımızı hem karşılaştırma hem de kopyalama olarak kabul etseydik dahi, algoritmanın belki bazı katsayıları değişebilir (3*n/2 yerine 5*n/2 ya da 6*n/2 vb.) ancak asimptotik olarak zaman karmaşıklığının dahil olduğu "order-of-growth" sınıfı değişmezdi.